Part 1 — Programming the data plane with DOCA Flow

Programming SmartNICs: From Packet Processing to Programmable Transport

ACM SIGCOMM 2026

Fernando Ramos Muhammad Shahbaz Mina Tahmasbi Arashloo Mario Baldi
Francisco Pereira Bilal Saleem Tyler Liu Chenxingyu Zhao

INESC-ID, Instituto Superior Técnico, University of Lisbon University of Michigan University of Waterloo
NVIDIA Purdue University University of Washington

In this part you program the **data plane** of a BlueField-3: you tell the NIC what to do with packets *in its own hardware, at line rate*, before any CPU sees them. You will complete a program that marks live RoCE traffic with an ECN congestion signal — the signal the congestion controller you write in Part 2 reacts to.

> **Prerequisites.** You are logged into the **Arm cores** of a BlueField-3 — a normal Ubuntu shell that happens to run inside the NIC. You have this repository in your home directory and can run sudo. **Every command below is typed on the Arm cores**; the host is never involved.

First, check which DOCA release your card runs, because the exercise ships in two versions:

```
$ cat /opt/mellanox/doca/applications/VERSION 2>/dev/null || cat /opt/mellanox/doca/VERSION
```

DOCA 2.7 or 2.9 means you work in `doca-2/`; DOCA 3.1 or 3.4 means `doca-3/`. This guide marks the few places they differ with **[2.x]** and **[3.x]**; everything unmarked applies to both.

Part A — The card, and getting traffic onto it

Your card has **two ports connected to each other**, so whatever leaves `p1` arrives at `p0`. That makes a single card behave like a small two-node network talking to itself. The two endpoints are lightweight virtual NICs called **SFs** (sub-functions), one per port, each in its own network namespace so the kernel cannot short-circuit them locally:

Endpoint	RDMA device	Namespace	IP	Role
SF on port 0	`mlx5_2`	`ns0`	`10.0.0.1`	**server** (receives)
SF on port 1	`mlx5_3`	`ns1`	`10.0.0.2`	**client** (sends)

The client sends RDMA WRITEs to the server. They leave `p1`, cross the cable, and arrive at `p0` — where **your program** sees them.

What you are reproducing is an everyday situation in a datacenter network: a sender is going as fast as it can, the network becomes congested, a switch on the path signals that congestion by setting the **ECN** bits in the IP header, and the sender slows down. Here, **your card plays the congested switch** — no congestion actually exists, you are *manufacturing* the signal, which is what makes it easy to see and to dial up and down.

That splits into the two halves of the tutorial:

- **Part 1, this guide.** Program the NIC to set the ECN "congestion experienced" (CE) mark on a fraction of the packets going past — you choose the fraction — and to copy a sample of them to a file so you can check your work. Everything happens in hardware; your program only installs the rules.
- **Part 2.** Program the DPA to *react*: the server's NIC answers a CE-marked packet with a congestion notification, and your algorithm turns each one into a lower send rate for the client.

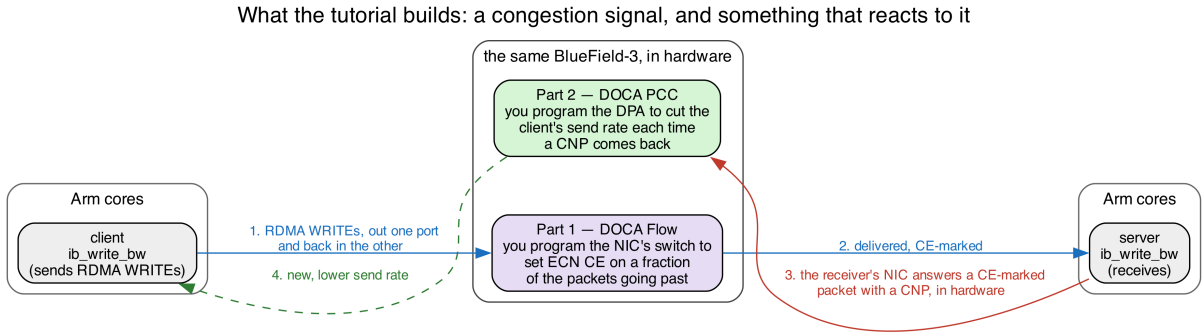

Figure 1: You write the marking in Part 1 and the reaction in Part 2. The client, the server, and the CNP the receiver's NIC sends back are already there.

Where things are. The repository has one directory per DOCA release, plus a matching `-solutions` directory holding the finished program:

```
doca-2/doca-flow/doca_flow_ecn_pcap.c              <- the file you edit
doca-2-solutions/doca-flow/doca_flow_ecn_pcap.c    <- the finished version
scripts/                                           <- traffic generators
```

Substitute `doca-3` throughout if that is your release. **Every command in this guide is run from the top of the repository**, so nothing below needs a `cd`.

Build it. This also confirms your toolchain works:

```
$ meson setup doca-2/build doca-2
$ ninja -C doca-2/build
```

`meson setup` is first-time only; after an edit, `ninja -C doca-2/build` on its own is enough.

Run it. It needs `sudo`, because it programs the NIC:

```
$ sudo ./doca-2/build/doca-flow/doca_flow_ecn_pcap -- --percent 100
```

The bare `--` is required: everything before it goes to the DPDK library, everything after it to our program.

Put traffic on the card, from a second shell:

```
$ ./scripts/benchmark.sh          # starts client + server, draws a live throughput chart
```

Ctrl-C stops both. If you would rather watch each side's raw output, run `./scripts/run_server.sh` and `./scripts/run_client.sh` in two shells instead — server first, since the client dials it.

> **This is the most important idea in the tutorial.** The instant a DOCA Flow program starts, **it takes ownership of the NIC's switch**. From then on the NIC forwards *only* what your pipes say to forward. So right now, with every function still empty, the program **blackholes everything**: `benchmark.sh` will not connect, and even `ping` between `ns0` and

`ns1` is 100% lost. Stop the program and the link comes straight back. That is not a bug — it is your starting point.

Part B — DOCA Flow in one page

DOCA Flow is how you program that switch from C. Your program builds a small graph of **pipes**, and each pipe answers four questions:

Part	The question it answers	Example
match	*which* packets does this pipe act on?	"all IPv4 packets"
monitor	*how many* packets hit it?	a hardware counter you can read
actions	*what* do we change in the packet?	rewrite a header field, or nothing
forward	*where* does the packet go next?	another pipe, a port, the CPU, or drop

You describe these rules once, at startup. The NIC then applies them to every packet in hardware, while your program sits idle printing counters. That is the whole idea: **match, count, modify, forward.**

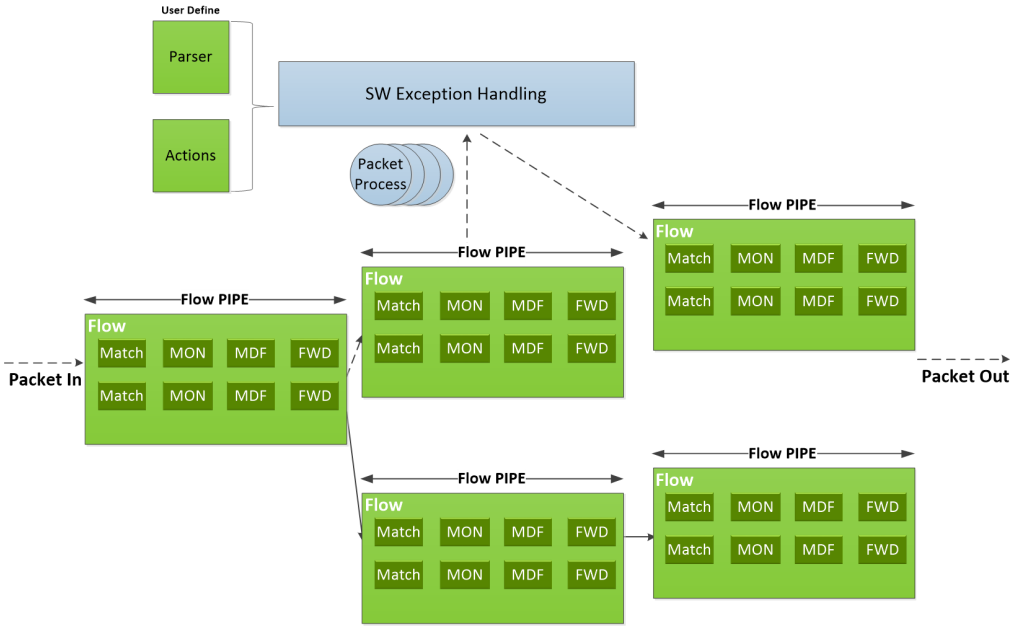

Figure 2: Pipes chain into a graph. Within a pipe, each entry is a Match, a monitor (MON), a modify/action stage (MDF) and a forward (FWD). Source: NVIDIA DOCA Flow programming guide, "Architecture".

Two things about pipes are worth getting straight before you write any, because both are easy to get backwards:

> **Pipe = which fields. Entry = what values.** A pipe is a *template*: writing `0xFF` into a field of its match means "this field participates", and `0x00` in the mask means "…but do not actually compare it". The **entry** you add afterwards supplies the value really compared. Creating a pipe installs no rule in the hardware; adding an entry does. The same split applies to actions — the pipe declares "entries may rewrite this field", the entry says "…to this value".
>
> **One pipe is the root.** Every packet entering the switch starts its lookup at the pipe marked `is_root`, and that is the only way into the rest of the graph. Until the root pipe exists, none of your other pipes are reachable no matter how correct they are — which is exactly why the untouched program forwards nothing.

You start from two worked examples. In `doca_flow_ecn_pcap.c`, `create_passthrough_pipe()` is a complete, minimal pipe and `create_to_cpu_pipe()` shows how to deliver packets to your own process.

Neither needs changing — read the first one before you write anything, because **every** pipe in the file, including all four of yours, has the same five-part shape:

1. **Create a pipe configuration** with `doca_flow_pipe_cfg_create()`, against the port it belongs to. What comes back is a builder, not a pipe.
2. **Describe it**, with one setter per property: `..._set_name()`, `..._set_type()`, `..._set_domain()`, `..._set_is_root()` and `..._set_nr_entries()`, plus `..._set_match()` for the match template — and `..._set_actions()` and `..._set_monitor()` when the pipe rewrites or counts. `..._set_match()` takes two `doca_flow_match` structs: the first says which fields participate, the second masks them.
3. **Create the pipe** with `doca_flow_pipe_create()`, passing two `doca_flow_fwd` structs — where a hit goes, and where a miss goes. Then discard the builder with `doca_flow_pipe_cfg_destroy()`; the pipe keeps no reference to it.
4. **Add the entries** with `doca_flow_pipe_add_entry()`, once each. This is the call that actually installs a rule — the pipe on its own does nothing.
5. **Install and check** with `doca_flow_entries_process()`, then confirm that every entry landed.

Step 5 is not optional. `doca_flow_pipe_add_entry()` returns as soon as the driver has taken the rule; the hardware's verdict arrives later, through a callback. A pipe that exists but installed nothing forwards nothing, silently — the hardest failure here to spot from the outside.

Part C — Build the pipeline

In `doca-2/doca-flow/doca_flow_ecn_pcap.c`, four function bodies are empty, marked `TODO 1` to `TODO 4`. Everything else is done. Laid out logically, the whole program is three phases, and only the middle one is yours (we use Python here purely as pseudo-code, to show the logical components of the template C file we provide you):

```
def main():
    setup_logging()
    parse_args()                        # --pcap, --percent, --sample
    open_capture_pcap()                 # only if --pcap was given

    # ---- bring-up: device and library. None of this is pipeline work ----
    dev = open_and_probe_dev(0)         # PF0, probed into DPDK with its SF representor
    configure_and_start_dpdk_port(dev)  # packet buffer pool, RX/TX queues
    initialize_doca_flow()              # switch mode, hardware steering, counters
    port = port_start(dev)              # the PF0 proxy port -- must be started first
    sf = rep_port_start(...)            # the server SF's representor

    build_pipeline(port, cfg, out)      # <-- ALL of your work is reached from here

    # ---- runtime ----
    run_capture_loop()                  # populates the pcap & print counters
    teardown()
```

Those are the real function names, so you can grep for any of them.

`build_pipeline()` is the one to read closely: it is the whole exercise in a single function, naming which pipes exist, in what order they are created, and how they chain.

```
def build_pipeline(port, cfg):
    capture = cfg.pcap_path is not None         # set by --pcap

    if capture:
        cpu = create_to_cpu_pipe(port)          # GIVEN -- an RSS pipe into your process
```

```
        bind_capture_mirror(port, cpu)          # TODO 1

    passthrough = create_passthrough_pipe(port) # GIVEN -- plain forward to the server SF

    # TODO 2, called twice: once forwarding plainly, once marking on the way through.
    pass_cap = create_forward_to_sf_pipe(port, mark=False, mirror=capture, miss=passthrough)
    if cfg.percent > 0:
        mark_cap = create_forward_to_sf_pipe(port, mark=True, mirror=capture,
↪   miss=passthrough)

    # Which pipe wire traffic enters, decided once at startup.
    if cfg.percent == 100:
        wire_target = mark_cap                  # mark everything
    elif cfg.percent == 0:
        wire_target = pass_cap                  # mark nothing
    else:
        wire_target = create_sampling_pipe(port, hit=mark_cap, miss=pass_cap)   # TODO 3

    create_root_pipe(port, wire_target)         # TODO 4
```

> **[3.x]** Two differences. `create_flood_pipe()` replaces `bind_capture_mirror()` as `TODO 1`, and is built *after* `passthrough` because it forwards to it. And `TODO 2` takes that flood pipe where 2.x takes a `capture` flag.

Now read the two conditionals. With `--percent 100` and no `--pcap`, `capture` is false and the percent test takes its first arm — so `TODO 1` and `TODO 3` are **never called**. That is what lets you get a working program from half the work, and it is why we do the four **out of numeric order**.

And here is the graph those calls produce. Keep it open while you work: every name in it is a function you are about to write, or one you have been given.

Stage 1 — mark and forward (`TODO 4`, then `TODO 2`)

Fill in two functions and leave the other two empty.

`create_root_pipe()` (`TODO 4`) — the root. Every packet hits it first, and it sorts by the port the packet arrived on: from the wire, into your marking pipe; coming back from the server, straight out to the wire. Two entries, keyed on the ingress port field (**[2.x]** `parser_meta.port_meta`, masked `UINT32_MAX`; **[3.x]** `parser_meta.port_id`, masked `UINT16_MAX`). Because the two entries go to different places, the pipe-level forward is `DOCA_FLOW_FWD_CHANGEABLE` and each entry brings its own. Set `is_root` true here and on no other pipe.

> **The return direction must not be marked.** It carries the RoCE acknowledgements and the congestion notifications, and marking those would corrupt the very feedback your Part 2 controller reacts to.

`create_forward_to_sf_pipe()` (`TODO 2`) — the marking pipe. Start from the `create_passthrough_pipe()` shape (match IPv4, forward to the server's SF), then add two things:

- a **counter**, via `monitor.counter_type = DOCA_FLOW_RESOURCE_TYPE_NON_SHARED`. This is what makes the `CE marked:` number move; without it you cannot tell whether anything is working.
- when `mark` is true, the **action that rewrites the ECN bits to CE**. Declare `outer.ip4.dscp_ecn` as `0xFF` in a pipe-level action template, and have the entry write the value. RFC 3168 gives the two-bit field as `Not-ECT 00`, `ECT(1) 01`, `ECT(0) 10`, `CE 11`, so the byte you want is `0x03`.

The function is called twice — once with `mark` false, once true — so everything mark-specific goes behind `if (mark)`. Match IPv4 *whatever ECN bits it arrived with*, using the wildcard idiom from Part B, and remember to reset that byte to `0x00` before adding the entry, since the same struct is reused as the entry's values. Finally, hand the installed entry back through `out_entry` — that is what the counter

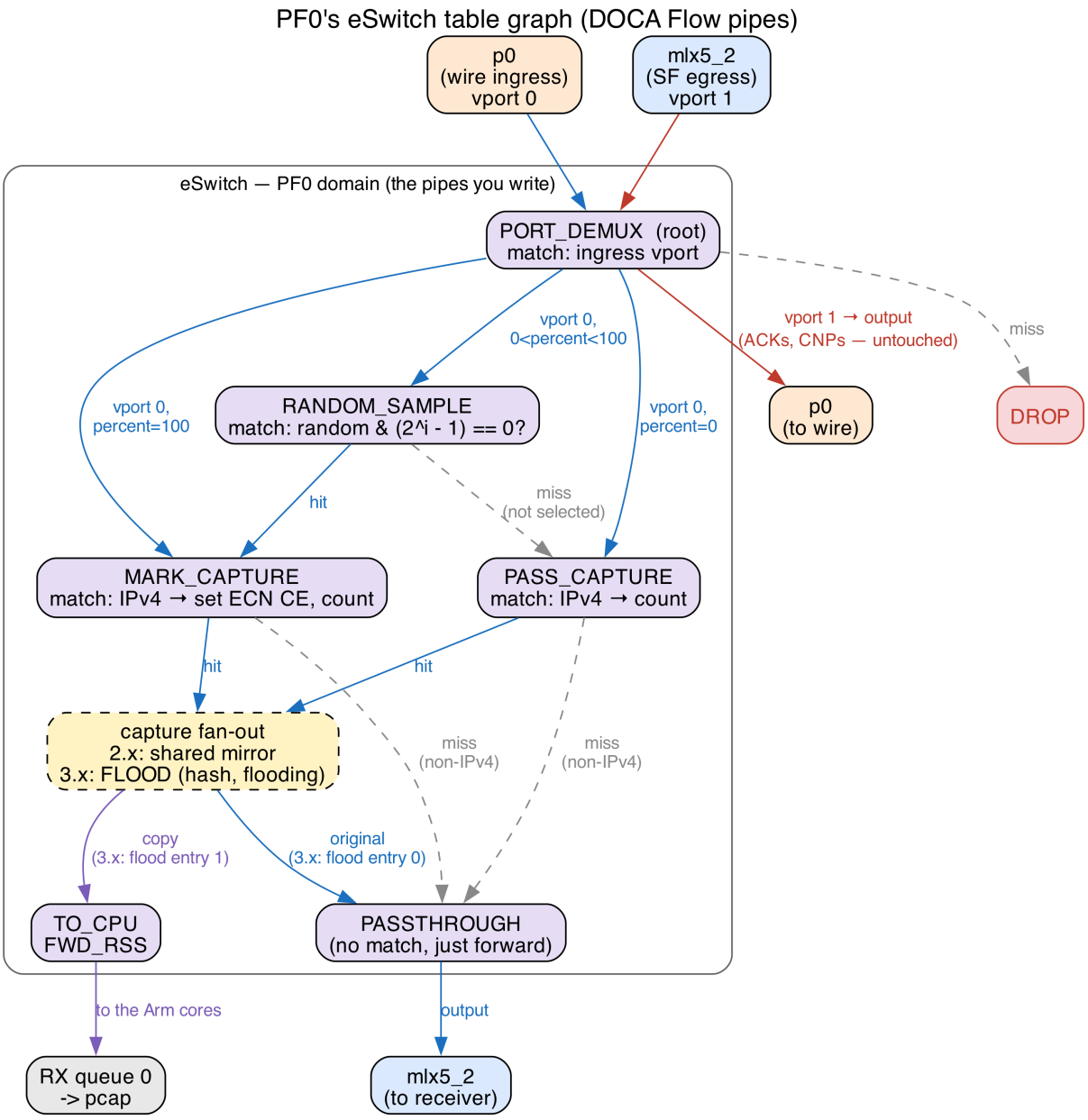

Figure 3: PF0's switch table graph. Only one of the three wire-ingress branches exists in a given run, chosen by `--percent`.

report queries. Leave the `mirror` / `flood_pipe` argument handling as it is for now; it does nothing until Stage 2.

Rebuild and run, with no `--pcap`, so the two functions you skipped are never reached:

```
$ ninja -C doca-2/build
$ sudo ./doca-2/build/doca-flow/doca_flow_ecn_pcap -- --percent 100
```

With `benchmark.sh` running, the counter should climb once a second and throughput should be back at line rate:

```
CE marked: 57060637, passthrough: 0 (100% marked)
```

`CE marked:` climbing means you are rewriting headers in hardware. You cannot *see* the bit yet — the counter only proves packets went through your marking pipe. That is what Stage 2 is for.

Stage 2 — capture a copy and see the mark (`TODO 1`)

Now send a *copy* of every packet to a capture file, so you can look at the ECN bits yourself. The original keeps forwarding to the server untouched. The mechanism differs between releases:

[2.x] **`bind_capture_mirror()`** builds no pipe. A mirror is a port-level *shared resource*: point its target's `fwd` at `cpu_pipe` — **this is where the copy goes** — then use `DOCA_TUT_MIRROR_SET_ORIG_FWD` for where the **original** carries on, then configure it under `MIRROR_ID` and *bind* it to the port. Configuring alone does nothing. Getting the copy wrong loses the capture; getting the original wrong black-holes your data path.

[3.x] **`create_flood_pipe()`** builds a `DOCA_FLOW_PIPE_HASH` pipe running the **flooding** algorithm, which delivers each packet to *all* of its entries instead of hashing it to one. The constant is `DOCA_FLOW_PIPE_HASH_MAP_ALGORITHM_FLOODING`. Exactly two entries, since a hash pipe's count must be a power of two: entry `0` to `production_pipe`, entry `1` to `capture_pipe`. Order matters — ordering is only guaranteed for entry 0, so the real data path goes there and the copy, where reordering costs nothing, takes entry 1. Clear the forward struct between the two entries.

Run it with a capture file this time:

```
$ sudo ./doca-2/build/doca-flow/doca_flow_ecn_pcap -- --pcap /tmp/out.pcap --percent 100
```

The counter line grows a capture half, and writing to the file starts **paused**:

```
CE marked: 57060637, passthrough: 0 (100% marked) | mirrored: 2743653 -> pcap: 0 [PAUSED]
```

`mirrored:` (**[3.x]**: `flooded:`) climbing means copies are reaching the CPU. Press **SPACE** to start writing, wait a few seconds, then Ctrl-C to flush and close the file cleanly. Now look at your mark:

```
$ ./scripts/check_ecn_bits_from_pcap.sh /tmp/out.pcap
#   tos 0x3,CE   <- a marked packet
#   (no tos)     <- an unmarked one; tcpdump omits the field when the byte is zero
```

At `--percent 100`, every IPv4 packet should read `CE` — and throughput should have stayed at line rate throughout, because the copy is made in hardware.

Stage 3 (optional) — mark only some packets (`TODO 3`)

`create_sampling_pipe()` splits traffic probabilistically, in hardware. The NIC stamps every packet with a random 16-bit value in `parser_meta.random`; match it against `0` under `mask` — already computed for you as a power of two minus one — and exactly 1 packet in `(mask + 1)` hits. Hits go to the marking pipe, misses to the non-marking one; "miss" here means *not selected*, not an error. The entry adds nothing to the template — both outcomes are decided by the pipe's own two forwards.

```
$ sudo ./doca-2/build/doca-flow/doca_flow_ecn_pcap -- --pcap /tmp/out.pcap --percent 50
```

The startup banner prints the fraction actually achieved, rounded down to a power of two. Check it against the mix of marked and unmarked packets in the capture.

Check your answer

The finished program sits next to yours. The diff should show **only your four function bodies**:

```
$ diff doca-2/doca-flow/doca_flow_ecn_pcap.c doca-2-solutions/doca-flow/doca_flow_ecn_pcap.c
```

Debugging tips

- **Read the *last* error line, not the first.** A failed pipe prints a wall of internal DOCA errors. The final `[CRT]...[doca_check] <name>: <reason>` line names the pipe, and that name is the function to open.
- **It runs, but nothing forwards** — you have not filled in the root pipe. With no root, none of the other pipes are reachable.
- **`CE marked:` stays at 0** — either your marking pipe has no counter, or the root pipe never sends wire traffic into it.
- **Throughput collapses** — the data path is going somewhere it should not. Re-check the forwards in `create_forward_to_sf_pipe()` and the two entries in `create_root_pipe()`.
- **The pcap stays empty** — capture starts **paused**; press SPACE. That needs a real terminal, so run the program in the foreground.
- **A new run says the device is busy** — only one DOCA Flow program can own the switch at a time. Stop the previous one; if it was killed hard, `sudo pkill -f doca_flow`.
- **A pipe fails to install** — check the status after `doca_flow_entries_process()`, per step 5 of the shape in Part B. Success from the add-entry call alone proves nothing.

What you built

You can now treat the card as a two-port loopback carrying real RoCE traffic; follow how match/count/-modify/forward pipes chain from a root pipe; and write a pipeline that matches IPv4, counts it, rewrites its ECN bits to CE, forwards it, and copies it to a capture file — all in hardware, at line rate, with your program idle. That CE mark is exactly the signal the congestion controller in Part 2 reacts to.

Appendix A — The BlueField, in more detail

Reference material. You do not need any of this to finish the exercise.

Interacting with a BlueField-3 typically entails interfacing with — and often independently programming — four different architectural components:

Where	What it is	Used for
Host x86	The server the card is plugged into	Not used at all in this tutorial
Arm cores	Cortex-A78 cores running their own Ubuntu on the card	Where you log in, build and run
NIC ASIC / eSwitch	The match-action switch inside the NIC	What you program in Part 1
DPA	Datapath Accelerator: a many-threaded RISC-V engine on the data path	The congestion controller, Part 2

The cards are configured in **DPU mode** (also called embedded-CPU-function ownership, ECPF): the Arm subsystem owns the NIC's resources, and the host x86 sees only a plain network device.

Functions: PFs, VFs and SFs

A **function** is what the NIC exposes so that software can send and receive packets through it. It is not a piece of software, but a slice of the NIC hardware, with its own queues, its own MAC address, and its own identity on the network. There are three kinds of functions:

- **PF** (*physical function*): the real PCIe device the NIC presents. A BlueField-3 has one PF per physical port. These are the functions that own the hardware.
- **VF** (*virtual function*): an SR-IOV slice of a PF, created so that a virtual machine can be handed something that looks like its own NIC. Not used in this tutorial.
- **SF** (*scalable function*, also called a *sub-function*): the same idea as a VF, but not tied to PCIe. An SF is created on the Arm with `mlnx-sf`, is much cheaper than a VF, and you can have far more of them. **This tutorial uses SFs** as its traffic endpoints.

Each function shows up on the Arm's Linux as **two** different devices:

Interface	What it is	Example
netdev	An ordinary Linux interface — what `ip link` lists and `ping` uses.	`p0`
RDMA device	The same function through the RDMA stack, bypassing the kernel.	`mlx5_0`

In this tutorial we use RDMA over Converged Ethernet (**RoCE**), which is RDMA carried inside ordinary UDP/IP packets, so we use the RDMA devices (`mlx5_N`). Because RoCE is just UDP/IP on the wire, the switch inside the NIC can match and rewrite it like any other packet.

Function	RDMA device	Netdev
PF0	`mlx5_0`	`p0`
PF1	`mlx5_1`	`p1`
SF on PF0	`mlx5_2`	`enp3s0f0s0`
SF on PF1	`mlx5_3`	`enp3s0f1s0`

The eSwitch

The eSwitch (embedded switch) is a hardware block inside the NIC that forwards packets between every function and every physical port on the card. Every packet that arrives from the wire and every packet

a function transmits passes through it, and it decides where that packet goes. It is the single most important component in this part of the tutorial, and we use DOCA Flow to program it.

Its rule table is called the **FDB** (forwarding database). Each PF (port) owns its own logical eSwitch domain, with its own rules, so programming PF0's eSwitch leaves PF1's forwarding untouched.

Every endpoint the eSwitch can send a packet to is a **vport** (virtual port): the physical ports are vports, and so is every PF, VF and SF. The eSwitch does not reach an SF directly, though. It reaches the SF's **representor** — a switch-side netdev that stands in for it. The two are the same function seen from opposite sides:

- from *inside* the namespace or VM using it, the SF appears as its netdev and its `mlx5_N` RDMA device — this is where an application sends and receives;
- from the *switch's* side, the same SF appears as its representor, which is what a forwarding rule names.

So "deliver this packet to the receiver" is written, in a rule, as "output to that representor's vport" — and the packet then pops out of the corresponding `mlx5_N` device inside the namespace. In NVIDIA's figure below, `rep0` and `rep1` are the representors of `VF0` and `VF1`; substitute SFs for VFs and it is our layout exactly, with the DOCA Flow application running on the Arm.

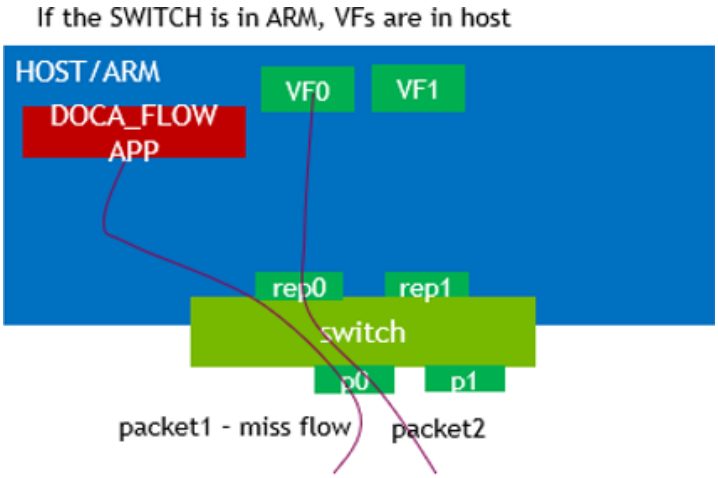

Figure 4: Applications use the functions (`VF0`, `VF1`); forwarding rules name their representors (`rep0`, `rep1`) and the physical ports. Source: NVIDIA DOCA Flow programming guide, "Domains in Switch Mode".

By default, each PF's vports are wired together by an OVS bridge with hardware offload — the "normal" forwarding path that makes the card behave like an ordinary NIC when nothing else is running. A DOCA Flow program **replaces** that for the PF it attaches to, from the moment it starts until it exits.

The whole round trip

The client issues an RDMA WRITE through `mlx5_3`; PF1's eSwitch — untouched by you — sends it out `p1`; it crosses the cable to `p0`; **PF0's eSwitch, your pipeline, marks and forwards it**; the server receives it via `mlx5_2`; the server's NIC answers a CE-marked packet with a **CNP** (Congestion Notification Packet) in hardware; the CNP travels back the same way; and on the client's side it raises an event on the DPA, where the Part 2 algorithm sets a new send rate.

Appendix B — DOCA Flow concepts in full

Reference material. Part B has the working subset.

Port. A DOCA Flow handle on one vport. In *switch mode* the PF uplink is also the **proxy port** (switch-manager port): it must be started first, and every pipe belongs to it — even pipes that forward somewhere else entirely.

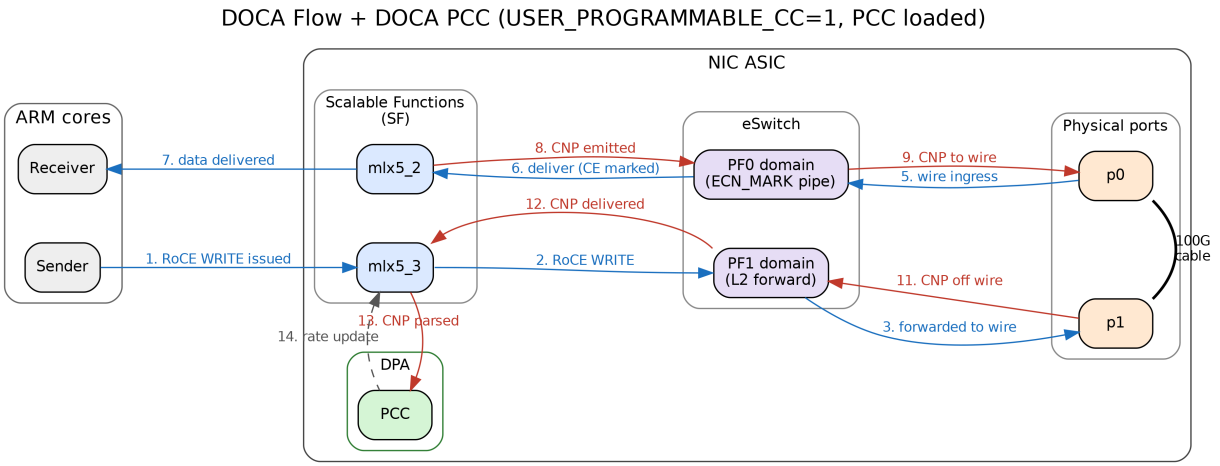

Figure 5: The end-to-end data path with both parts running.

Pipe. A flow table, not a pipeline stage. NVIDIA's wording: a pipe "is a template that defines packet processing without adding any specific hardware rule".

Pipe type	What it does
`BASIC`	Ordinary match-action table
`CONTROL`	Entries carry priorities (0-7), resolving conflicts between overlapping matches
`LPM`	Longest-prefix match, for routing-table-shaped lookups
`ACL`	Five-tuple matching with masks
`ORDERED_LIST`	A fixed sequence of actions per entry, when their order matters
`HASH`	Selects an entry by an index computed from a hash, rather than by matching fields

Entry. A concrete rule inside a pipe.

Match. Each field is *ignored* (left zero), *constant* (set in the pipe, the same for all entries), or *changeable* (all-ones placeholder in the pipe, real value supplied per entry).

Actions. Mostly field rewrites — MAC, IP, DSCP/ECN, L4 ports, metadata — but also encapsulation and decapsulation. Matches and actions are not alternatives to one another: every entry has a match, and *may additionally* have actions, a monitor and a forward.

Monitor. Counters, meters, and (on 2.x) mirroring.

Forward. Where the packet goes when the lookup finishes. Each pipe has a *hit* forward and optionally a *miss* forward:

Forward	Meaning
`DOCA_FLOW_FWD_PORT`	Output to a vport: a physical port, or a function's representor
`DOCA_FLOW_FWD_PIPE`	Jump to another pipe — this is how pipes chain
`DOCA_FLOW_FWD_RSS`	Deliver to a receive queue of your program, **on the Arm cores**
`DOCA_FLOW_FWD_DROP`	Discard
`DOCA_FLOW_FWD_CHANGEABLE`	Each entry brings its own forward
`DOCA_FLOW_FWD_HASH_PIPE`	Jump to a hash pipe, naming the algorithm to run (3.x)

Shared resources. Objects living on the port rather than inside one pipe, so several pipes can point at a single instance: meters, counters, RSS contexts, encap/decap contexts — and, on 2.x, **mirrors**. Mirrors were removed in DOCA 3.2, which is the largest difference between the two versions of this exercise.

Parser metadata. `parser_meta` holds values the hardware parser attaches to each packet, rather than header fields: `outer_l3_type` (what the parser saw), `random` (a fresh 16-bit value per packet, which is what makes hardware sampling possible), and the ingress port.

Note that `outer.ip4.dscp_ecn` is the **whole** TOS byte, so writing `0x03` sets ECN to CE *and* clears DSCP. Our traffic carries DSCP 0, so it makes no difference here.

Appendix C — The API calls used in this exercise

The headers on the card are the authority: `/opt/mellanox/doca/include/doca_flow.h`, *with packet field types in* `doca_flow_net.h`. *Every call is documented there. This is only a map of which ones matter here.*

`grep -n 'dscp_ecn\|parser_meta' /opt/mellanox/doca/include/doca_flow*.h` answers most structural questions faster than the web documentation. If you are on VS Code Remote-SSH, `.vscode/c_cpp_properties.json` already points IntelliSense at these paths, so "go to definition" works.

Call	What it is for
`doca_flow_pipe_cfg_create` / `_destroy`	Start and finish describing a pipe
`doca_flow_pipe_cfg_set_name`	Name it — this is what error messages report
`doca_flow_pipe_cfg_set_type`	`BASIC`, `HASH`, …
`doca_flow_pipe_cfg_set_domain`	Which steering domain; always `..._DOMAIN_DEFAULT` here
`doca_flow_pipe_cfg_set_is_root`	Mark the one root pipe
`doca_flow_pipe_cfg_set_nr_entries`	How many entries you will add
`doca_flow_pipe_cfg_set_match`	The match template and its mask
`doca_flow_pipe_cfg_set_actions`	The action template(s)
`doca_flow_pipe_cfg_set_monitor`	Counter, meter, and **[2.x]** the mirror id
`..._cfg_set_hash_map_algorithm`	**[3.x]** flooding, for the capture copy
`doca_flow_pipe_create`	Create the pipe, with its hit and miss forwards
`doca_flow_pipe_add_entry`	Add an entry — **[2.x]** and 3.1
`doca_flow_pipe_basic_add_entry`	Add an entry — **[3.x]**; takes the action index as an argument
`doca_flow_pipe_hash_add_entry`	Add an entry to a hash pipe, by index
`doca_flow_entries_process`	Drive queued entries to completion, then check the status
`doca_flow_resource_query_entry`	Read an entry's counter (already written, in `query_pkts()`)
`doca_flow_shared_resource_set_cfg`	**[2.x]** configure the mirror
`doca_flow_shared_resources_bind`	**[2.x]** bind it to the port — configuring alone does nothing

Version differences

	[2.x]	**[3.x]**
copy to the pcap	shared mirror, via `monitor.shared_mirror_id`	`DOCA_FLOW_PIPE_HASH` + `..._ALGORITHM_FLOODING`
entry install	`doca_flow_pipe_add_entry`; action index inside `doca_flow_actions`	`doca_flow_pipe_basic_add_entry`; action index as an argument
entry flags	`DOCA_FLOW_NO_WAIT`, `DOCA_FLOW_WAIT_FOR_BATCH`	`DOCA_FLOW_ENTRY_FLAGS_NO_WAIT`, `..._WAIT_FOR_BATCH`
RSS forward	flat `fwd.rss_queues` / `num_of_queues` / `rss_outer_flags`	nested `fwd.rss.queues_array` / `nr_queues` / `outer_flags`
ingress port match	`parser_meta.port_meta` (`uint32`)	`parser_meta.port_id` (`uint16`)

`doca_flow_compat.h`, force-included by the build, already smooths the 3.1-versus-3.4 seam inside the `doca-3` tree. It does **not** bridge 2.x to 3.x.

The program's own flags

Flag	Meaning
`--percent N`	CE-mark this share of packets, `[0, 100]`, rounded down to a power-of-two fraction. Default 100.
`--pcap FILE`	Also capture a copy of the traffic to `FILE`. Omit for pure marking mode.
`--sample N`	Write only about 1-in-N captured packets to the file. Marking and forwarding are unaffected.

Further reading

- DOCA Flow programming guide — swap the version in the URL to match your card, e.g. `.../archive/2-9-0/doca-flow`.
- BlueField modes of operation
- BlueField scalable function user guide
- NVIDIA ships around 20 single-concept sample programs on the card. They live under `/opt/mellanox/doca/samples/doca_flow/` — one `.c` file plus a README each, every one isolating a single idea.